System Design — Day 1

Topic: Scaling Basics (Vertical vs Horizontal, Load Balancers, Stateless vs Stateful)

1. Vertical Scaling

Making a single server bigger — more RAM, faster CPU, more disk.
Pro: Simple, no code changes needed.
Con: Hard physical ceiling — there's a limit to how big one single machine can get, no matter how much money you throw at it. Also a single point of failure.

2. Horizontal Scaling

Adding more servers instead of a bigger one — traffic is spread across many machines.
Pro: Near-infinite ceiling, no single point of failure.
Con: Needs something to distribute traffic across all the servers.

Horizontal scaling only actually works if two conditions are met:

- A load balancer to route traffic
- Stateless servers so any server can handle any request

3. Load Balancer

Sits in front of all the servers and is the single entry point for user traffic. It never does the actual work itself — it just routes each incoming request to one of the available servers (using rules like round-robin or least-connections).

4. Stateless vs Stateful Services

Stateless: The server doesn't remember anything between requests. Any server can handle any request — this is exactly why horizontal scaling works.
Stateful: The server holds onto data locally (e.g. "upload in progress, 60% done") in its own memory. This breaks horizontal scaling: the load balancer doesn't guarantee a user's next request lands on the same server, so if that data only exists on Server A's memory, Server B has no idea it exists.

5. Where Should Stateful Data Actually Live?

The fix for stateful data is to move it out of any single server's local memory into something all servers can reach. But different kinds of data belong in different places:

Data type	Where it lives	Why
"Upload 60% done" — temporary, fast-changing	Redis (in-memory store)	Speed — doesn't need to survive long
user_id, caption, upload_time — structured, permanent	Database (Postgres/MySQL)	Needs to be queried and must persist
The actual photo file — large, unstructured	Object storage (e.g. Amazon S3)	Cheap, scalable storage for big binary files

6. Key Takeaway

These 4 primitives — scaling, load balancers, stateless design, and the right storage for the right data type — are the building blocks that every larger system design question (Twitter, Uber, Netflix, etc.) is built from. Master these deeply instead of memorizing whole system diagrams.